UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

TUGAS 2

Postman Collection dan Automated Test Script

Pengujian *RESTful API* dengan *Postman Automation*

Pengembangan Aplikasi Web II | SUB-CPMK-7

Nama dan NIM	M Hafizh Muhyiddin 2428240127
Kelas	SI5C
Judul project / topik API	Blog
Tautan repositori	https://github.com/hadinnn/tugas1-restful-2428240127
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	19 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah laporan **Tugas 2**: menguji RESTful API project akhir Anda dengan Postman. Seluruh isinya menggunakan **API milik Anda sendiri**, bukan API Mahasiswa pada contoh materi. Pelajari langkahnya pada materi Pertemuan 8 sebelum mengisi.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]`, dengan data API Anda.
2. Bukti berupa gambar (*screenshot*) diisi pada kotak bergaris putus-putus. Cuplikan kode test script **ditempel sebagai teks**, bukan gambar.
3. Setiap bukti memiliki keterangan dan kotak **Penjelasan**. Tulis penjelasan dengan kalimat Anda sendiri dan sebut nama *request* atau fungsi dari API Anda.
4. Simpan hasil akhir sebagai PDF, lalu kumpulkan melalui LMS sesuai ketentuan dosen (format nama berkas dan batas waktu mengikuti petunjuk di LMS).

B. Bukti yang diminta

No	Bukti	Bagian	Jenis
1	Daftar request dalam Collection	Bab 2	Screenshot (wajib)
2	Cuplikan test script	Bab 3	Kode teks (wajib)
3	Ringkasan Collection Runner	Bab 4	Screenshot (wajib)
4	Hasil Newman di terminal	Bab 5	Screenshot (pengayaan)

C. Syarat tangkapan layar

Bukti	Yang harus terlihat
Collection	Nama Collection, kedua folder (bila ada), dan seluruh nama request. Buka semua folder sebelum mengambil gambar.
Ringkasan Runner	Nama Collection, Environment yang dipakai, jumlah request, jumlah test **lulus** dan **gagal**. Seluruh test harus lulus (0 gagal).
Newman	Perintah yang diketik, daftar hasil, dan tabel `executed` dan `failed` di bagian akhir.
Larangan	Gambar dipotong sehingga angka hasil tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Sebelum mengambil screenshot Runner
Nyalakan ulang server agar data kembali ke kondisi awal, pilih Environment yang benar, lalu jalankan seluruh Collection. Bila ada test gagal, perbaiki dulu dan jalankan ulang sampai 0 gagal.

Bab 1 Identitas API dan Daftar Endpoint

Bab ini menjelaskan API yang diuji, agar pembaca memahami cakupan pengujian pada bab berikutnya.

1.1 Deskripsi singkat API

Penjelasan:
API ini mengusung topik Blog dengan entitas utama berupa artikel yang berfungsi untuk mengolah data postingan tulisan. Pengembangan sistemnya menggunakan Node.js dan Express.js dengan struktur sederhana yang ditulis langsung dalam file app.js. Seluruh alur kerja dan fungsionalitas *endpoint*-nya juga telah diuji hingga berhasil menggunakan Postman.

1.2 Daftar endpoint

Tuliskan **seluruh** *endpoint* API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey` atau mekanisme serupa.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/articles		200	Menampilkan seluruh data blog artikel
2	GET	/articles/:id		200	Menampilkan satu data blog artikel per ID
3	POST	/articles	Ya	201	Menambahkan satu data blog artikel
4	PUT	/articles/:id	Ya	200	Mengubah satu data blog artikel per ID
5	DELETE	/articles/:id	Ya	204	Menghapus satu data blog artikel per ID

1.3 Variabel Environment

Tuliskan variabel yang Anda simpan di Environment. Jangan menuliskan nilai rahasia sungguhan.

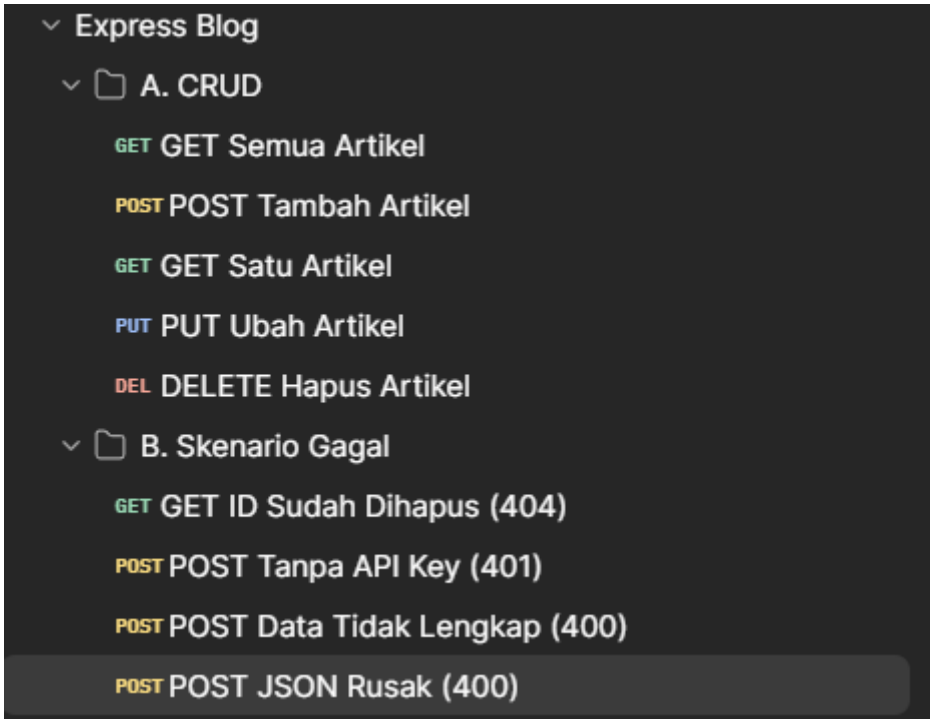
No	Nama variabel	Nilai (contoh)	Fungsi
1	base_url	http://localhost:3000	Alamat server
2	api_key	kunci-rahasia-paw2	Nilai header kunci API
3	articlesId	(kosong, diisi script)	ID dari respons POST

Bab 2 Postman Collection

Bab ini membuktikan bahwa seluruh *endpoint* sudah dikelompokkan dalam satu Collection.

2.1 Daftar request dalam Collection

Harus terlihat pada screenshot: nama Collection dan seluruh request di dalam folder. Buka semua folder.



Gambar 1. Collection beserta seluruh request

Penjelasan:
Pengelompokan *request* dibagi menjadi folder A. CRUD untuk menguji fitur utama dalam kondisi normal (skenario sukses), serta folder B. Skenario Gagal untuk menguji penanganan *error* dan validasi aplikasi. Urutannya disusun logis mulai dari alur positif dulu agar dipastikan semua fungsi dasar berjalan lancar, lalu dilanjutkan ke alur negatif untuk membuktikan sistem dan *middleware* mampu menolak *request* yang bermasalah secara benar.

2.2 Tabel request

Salin daftar request pada gambar di atas ke tabel berikut. Setiap *endpoint* pada Bab 1 harus punya minimal satu request.

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
1	GET Semua Artikel	GET	{{base_url}}/articles	A. CRUD
2	POST Tambah Artikel	POST	{{base_url}}/articles	A. CRUD
3	GET Satu Artikel	GET	{{base_url}}/articles/{{articlesId}}	A. CRUD
4	PUT Ubah Artikel	PUT	{{base_url}}/articles/{{articlesId}}	A. CRUD

No	Nama request	Metode	Alamat (pakai {{base_url}})	Folder
5	DELETE Hapus Artikel	DELETE	{{base_url}}/articles/{{articlesId}}	A. CRUD
6	GET ID Sudah Dihapus (404)	GET	{{base_url}}/articles/{{articlesId}}	B. Skenario Gagal
7	POST Tanpa API Key (401)	POST	{{base_url}}/articles	B. Skenario Gagal
8	POST Data Tidak Lengkap (400)	POST	{{base_url}}/articles	B. Skenario Gagal
9	POST JSON Rusak (400)	POST	{{base_url}}/articles	B. Skenario Gagal

Bab 3 Automated Test Script

Bab ini menunjukkan *assertion* yang Anda tulis. Setiap request wajib memiliki minimal satu *assertion* status code dan satu *assertion* struktur atau isi data JSON.

3.1 Rekap assertion per request

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

1	GET Semua Artikel	200 OK	pm.test('Status code adalah 200', *function* () {
			  pm.response.to.have.status(200);
			});
			
			pm.test('Response berupa array', *function* () {
			  const jsonData = pm.response.json();
			  pm.expect(jsonData).to.be.an('array');
			});
			
			pm.test('Setiap data memiliki id dan judul', *function* () {
			  const jsonData = pm.response.json();
			  jsonData.forEach((*item*) => {
			  pm.expect(item).to.have.property('id');
			  pm.expect(item).to.have.property('judul');
			  });
			});
			
			pm.test('Response time di bawah 500ms', *function* () {
			  pm.expect(pm.response.responseTime).to.be.below(500);
			});
			
			pm.test('Data artikel pertama berjudul Mengenal REST API', *function* () {
			  const jsonData = pm.response.json();
			  pm.expect(jsonData[0].judul).to.eql('Mengenal REST API');
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

2	POST Tambah Artikel	201 Created	{
			"id": 3,
			"judul": "Ambisi Presiden di tahun 2026",
			"isi": "Presiden Republik Indonesia Prabowo Subianto memiliki tujuan ...",
			"penulis": "Buna Teddy",
			"kategori": "Politik",
			"dipublikasikan": false
			}
			
			pm.test('Status code adalah 201', *function* () {
			pm.response.to.have.status(201);
			});
			
			pm.test('Data baru memiliki id dan judul sesuai kiriman', *function* () {
			const jsonData = pm.response.json();
			pm.expect(jsonData).to.have.property('id');
			pm.expect(jsonData.judul).to.eql('Ambisi Presiden di tahun 2026');
			});
			
			if (pm.response.code === 201) {
			pm.environment.set('articlesId', pm.response.json().id);
			}

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

3	GET Satu Artikel	200 OK	pm.test('Status code adalah 200', *function* () {
			  pm.response.to.have.status(200);
			});
			
			pm.test('Data yang diambil adalah artikel yang berjudul Ambisi Presiden di tahun 2026', *function* () {
			  const jsonData = pm.response.json();
			  pm.expect(jsonData.id).to.eql(*Number*(pm.environment.get('articlesId')));
			  pm.expect(jsonData.judul).to.eql('Ambisi Presiden di tahun 2026');
			});

4	PUT Ubah Artikel	200 OK	{ "kategori": "Pemerintahan" }
			
			pm.test('Status code adalah 200', *function* () {
			  pm.response.to.have.status(200);
			});
			
			pm.test('Jurusan berubah, nama tetap', *function* () {
			  const jsonData = pm.response.json();
			  pm.expect(jsonData.kategori).to.eql('Pemerintahan');
			  pm.expect(jsonData.judul).to.eql('Ambisi Presiden di tahun 2026');
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

5	DELETE Hapus Artikel	204 No Content	pm.test('Status code adalah 204', function () {
			
			pm.response.to.have.status(204);
			});
			
			pm.test('Response tidak memiliki isi', function () {
			
			pm.expect(pm.response.text()).to.eql('');
			});

6	GET ID Sudah Dihapus (404)	404 Not Found	pm.test('Status code adalah 404', *function* () {
			  pm.response.to.have.status(404);
			});
			
			pm.test('Pesan error: Data tidak ditemukan', *function* () {
			  pm.expect(pm.response.json().message).to.eql('Data tidak ditemukan');
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

7	POST Tanpa API Key (401)	401 Unauthorized	{
			"judul": "Artikel Tanpa Key",
			"isi": "Tanpa Key",
			"penulis": "Tanpa Key",
			"kategori": "Politik",
			"dipublikasikan": true
			}
			
			pm.test('Status code adalah 401', *function* () {
			pm.response.to.have.status(401);
			});
			
			pm.test('Pesan error: API key tidak valid', *function* () {
			pm.expect(pm.response.json().message).to.eql('API key tidak valid');
			});

8	POST Data Tidak Lengkap (400)	400 Bad Request	{ "judul": "Hanya Judul" }
			
			pm.test('Status code adalah 400', function () {
			
			pm.response.to.have.status(400);
			});
			
			pm.test('Pesan error menyebut field wajib', function () {
			
			pm.expect(pm.response.json().message).to.include('wajib diisi');
			});

No	Nama request	Assertion status code	Assertion struktur atau isi JSON

9	POST JSON Rusak (400)	400 Bad Request	{"judul": "Rusak",}
			
			pm.test('Status code adalah 400', function () {
			
			pm.response.to.have.status(400);
			});
			
			pm.test('Respons tetap berupa JSON, bukan HTML', function () {
			pm.response.to.be.json;
			
			pm.expect(pm.response.json().message).to.eql('Format JSON tidak valid');
			});

3.2 Cuplikan 1: request GET

Tempel test script **satu request GET** pilihan Anda.

```
pm.test('Status code adalah 200', function () {
  pm.response.to.have.status(200);
});

pm.test('Response berupa array', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.be.an('array');
});

pm.test('Setiap data memiliki id dan judul', function () {
  const jsonData = pm.response.json();
  jsonData.forEach((item) => {
    pm.expect(item).to.have.property('id');
    pm.expect(item).to.have.property('judul');
  });
});

pm.test('Response time di bawah 500ms', function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test('Data artikel pertama berjudul Mengenal REST API', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData[0].judul).to.eql('Mengenal REST API');
});
```

Penjelasan:
Pengujian pertama berupa pengecekan kode status HTTP 200 penting karena menjamin server berhasil memproses permintaan pengambilan data tanpa kendala. Pengujian kedua berupa pengecekan tipe data utama berbentuk *array* penting karena *endpoint* ini mengambil banyak data sekaligus sehingga *frontend* butuh struktur tersebut untuk *looping*. Pengujian ketiga berupa pemeriksaan keberadaan properti id dan judul pada setiap objek artikel penting karena menjaga konsistensi struktur data agar *frontend* tidak mengalami *error* atau *undefined*. Pengujian keempat berupa pengukuran durasi balasan server di bawah 500ms penting karena memastikan performa API tetap cepat dan responsif bagi pengguna. Pengujian kelima berupa pengecekan isi judul artikel pertama secara spesifik penting karena memastikan integritas data dan kebenaran urutan data yang dikirim oleh server.

3.3 Cuplikan 2: request POST atau request yang menyimpan variabel

Tempel test script **request POST** (atau request lain yang menyimpan variabel ke Environment).

```
pm.test('Status code adalah 201', function () {
  pm.response.to.have.status(201);
});

pm.test('Data baru memiliki id dan judul sesuai kiriman', function () {
  const jsonData = pm.response.json();
  pm.expect(jsonData).to.have.property('id');
  pm.expect(jsonData.judul).to.eql('Ambisi Presiden di tahun 2026');
});

if (pm.response.code === 201) {
  pm.environment.set('articlesId', pm.response.json().id);
}
```

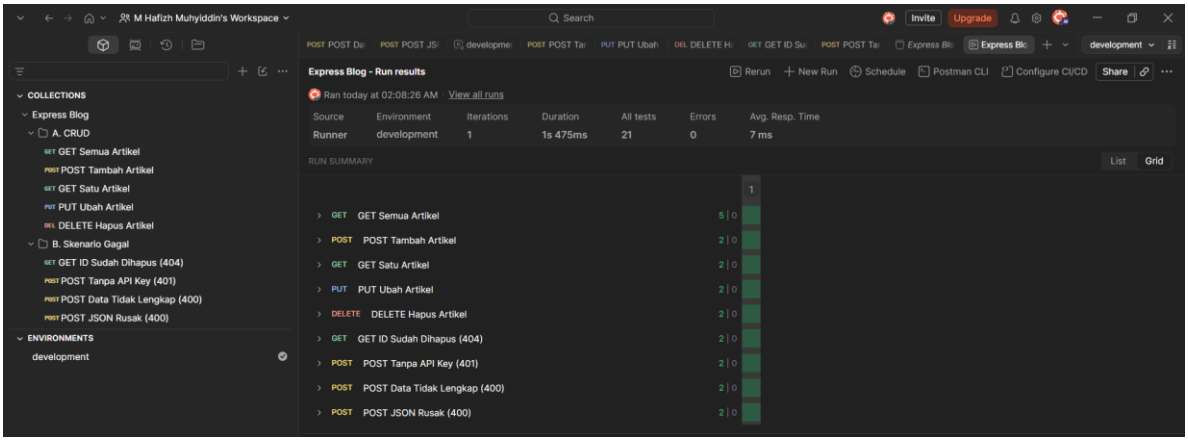
Penjelasan:
Data id dari respon artikel baru disimpan secara otomatis ke variabel *environment* articlesId melalui *script* pm.environment.set('articlesId', pm.response.json().id) saat status respon bernilai 201. Data ID ini selanjutnya digunakan oleh *request* GET Satu Artikel, PUT Ubah Artikel, dan DELETE Hapus Artikel untuk memanggil artikel tersebut. Jika urutan *request* ditukar (misalnya GET Satu Artikel dijalankan duluan sebelum POST Tambah Artikel), pengujian berikutnya akan gagal atau *error* 404 Not Found karena variabel articlesId belum terbentuk atau nilainya masih kosong.

Bab 4 Hasil Collection Runner

Bab ini adalah bukti utama bahwa seluruh test lolos saat Collection dijalankan sekaligus.

4.1 Ringkasan Collection Runner

Harus terlihat pada screenshot: nama Collection, Environment, jumlah request, jumlah test lulus dan gagal. Seluruh test harus lulus.



Gambar 2. Ringkasan hasil Collection Runner

Penjelasan:
Pengujian eksekusi Runner ini menjalankan sebanyak 9 *request* dengan total 21 *test*, dan seluruh pengujian tersebut berhasil tanpa ada *error* sama sekali. Sebelum menjalankan Runner, server Node.js/Express dinyalakan ulang (*restart*) terlebih dahulu agar data artikel di dalam memori kembali bersih ke kondisi awal, sehingga seluruh alur pengujian skenario sukses maupun gagal bisa berjalan secara konsisten dan valid dari awal tanpa terganggu sisa data dari pengujian sebelumnya.

4.2 Rekap angka hasil

Nama Collection	Express Blog
Environment yang dipakai	development
Jumlah request dijalankan	9
Jumlah test lulus	21
Jumlah test gagal	0
Tanggal dan jam pengujian	19 Oktober 2026 02:14 AM

4.3 Temuan selama pengujian

Catat kegagalan yang pernah muncul sebelum akhirnya lulus. Bagian ini menunjukkan bahwa pengujian benar-benar membantu menemukan masalah. Bila tidak ada, tulis "tidak ada kegagalan".

No	Request	Kegagalan (pesan)	Penyebab	Perbaikan
1	Tidak ada Kegagalan			

Bab 5 Newman di Terminal (Pengayaan)

> **Bagian ini opsional**
> Bab ini adalah pengayaan dan tidak mengubah kriteria penilaian. Isi bila Anda menjalankan Collection dengan Newman. Bila tidak, tulis "tidak dikerjakan" di bawah judul ini.

Tidak dikerjakan

5.1 Hasil Newman

Harus terlihat pada screenshot: perintah `newman run ...`, daftar hasil, dan tabel `executed` dan `failed`.

[Tempel screenshot di sini]

Gambar 3. Hasil menjalankan Collection dengan Newman

Gambar 3. Hasil menjalankan Collection dengan Newman

Penjelasan:

Tulis perintah yang Anda jalankan dan jelaskan arti exit code (0 atau 1) pada hasil Anda.

Bab 6 Repositori dan Berkas

Tautan repositori	https://github.com/hadinnn/tugas1-restful-2428240127
Lokasi berkas Collection (.json)	
Lokasi berkas Environment (.json)	
Berkas Environment tidak memuat kunci rahasia sungguhan	[ ] Ya [ ] Tidak
Repositori dapat dibuka dosen	[ ] Ya [ ] Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan pengalaman Anda mengerjakan tugas ini.

Pertanyaan 1

Dari seluruh *assertion* yang Anda tulis, mana yang paling berguna untuk menemukan kesalahan pada API Anda? Berikan alasannya.

Penjelasan:
Assertion yang paling bermanfaat selama pengerjaan tugas ini adalah pengujian validasi struktur data (to.have.property) untuk memastikan keberadaan properti utama seperti id dan judul. Pengujian ini sangat krusial karena status respons HTTP 200 OK tidak menjamin bahwa isi *payload* JSON sudah sesuai dengan skema yang diharapkan. Melalui pengujian ini, saya dapat secara otomatis memverifikasi integritas data yang dihasilkan oleh *controller* tanpa perlu melakukan pengecekan manual.

Pertanyaan 2

Apa perbedaan yang Anda rasakan antara menguji API secara manual dan memakai Collection Runner? Bagian mana yang paling sulit saat menulis test script, dan bagaimana Anda mengatasinya?

Penjelasan:
Perbedaan utama yang saya rasakan adalah efisiensi waktu, di mana pengujian manual mengharuskan pemeriksaan *request* satu per satu, sedangkan Collection Runner mampu mengeksekusi seluruh skenario pengujian secara otomatis hanya dalam sekali klik. Bagian paling menantang saat menyusun *test script* adalah menyesuaikan sintaks dan logika pengujian dari materi perkuliahan agar sesuai dengan entitas artikel pada API Blog. Saya mengatasinya dengan menganalisis bentuk respons JSON terlebih dahulu, kemudian menyesuaikan variabel serta penamaan properti seperti id dan judul pada skrip *assertion*.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan data API saya	✓
2	Setiap endpoint API saya ada di Collection	✓
3	Alamat memakai `{{base_url}}`, bukan ditulis langsung	✓
4	Setiap request punya assertion status code dan struktur JSON	✓
5	Nama setiap test berupa kalimat yang menyatakan harapan	✓
6	POST menyimpan ID dengan pemeriksaan status, dan request berikutnya memakainya	✓
7	Server dinyalakan ulang sebelum Runner dijalankan	✓
8	Screenshot Runner menampilkan 0 test gagal dan angkanya terbaca jelas	✓
9	Berkas Environment tidak memuat kunci rahasia sungguhan	✓
10	Berkas disimpan sebagai PDF dan dikumpulkan sesuai ketentuan LMS	✓

Lampiran B Kriteria Penilaian

Kriteria berikut sesuai dengan slide materi Pertemuan 8. Bobot nilai mengikuti ketentuan dosen di LMS.

Kriteria	Deskripsi	Dinilai dari
Kelengkapan Postman Collection	Seluruh endpoint project akhir tercakup dalam satu Collection	Bab 1 dan Bab 2
Ketepatan automated test script	Assertion status code dan struktur response JSON tepat sasaran	Bab 3
Keberhasilan Collection Runner	Seluruh test lolos saat Collection dijalankan	Bab 4
Kejelasan dokumentasi hasil	Tangkapan layar ringkasan pengujian disertakan dengan jelas	Bab 4 dan seluruh dokumen